

3.3 Guía de Funciones: LDA y Evaluación de Modelos

R — MASS + caret

```
library(MASS) # lda, qda
library(caret)
library(pROC)

data(iris)
set.seed(42)
idx <- createDataPartition(iris$Species, p=0.8, list=FALSE)
train <- iris[idx,]; test <- iris[-idx,]

# LDA
# ———
modelo_lda <- lda(Species ~ ., data=train)
print(modelo_lda)
# → Prior probabilities: P(clase) en train
# → Group means: media de cada feature por clase
# → Coefficients of linear discriminants: pesos de las combinaciones lineales

# Predicción
pred_lda <- predict(modelo_lda, test)
pred_lda$class # clases predichas
pred_lda$posterior # probabilidades a posteriori por clase
pred_lda$x # scores en el espacio LDA (coordenadas proyectadas)

# Matriz de confusión
confusionMatrix(pred_lda$class, test$Species)

# Visualizar proyección LDA
scores_df <- data.frame(
  LD1 = pred_lda$x[,1],
```

```

LD2 = pred_lda$x[,2],
Species = test$Species
)
library(ggplot2)
ggplot(scores_df, aes(x=LD1, y=LD2, color=Species)) +
  geom_point(size=3, alpha=0.8) +
  stat_ellipse(level=0.95) +
  labs(title="Proyección LDA - iris") +
  theme_minimal()

# QDA (cuando covarianzas son distintas por clase)
modelo_qda <- qda(Species ~ ., data=train)
pred_qda <- predict(modelo_qda, test)$class
confusionMatrix(pred_qda, test$Species)

# EVALUACIÓN RIGUROSA con CV
# -----
ctrl <- trainControl(
  method = "cv",
  number = 10,
  savePredictions = "final",
  classProbs = TRUE,
  summaryFunction = multiClassSummary # para multiclase
)

# Comparar KNN, NB, LDA, RL en mismos folds
set.seed(100)
m_knn <- train(Species~., data=iris, method="knn", trControl=ctrl, preProcess=c("center","scale"))
m_nb <- train(Species~., data=iris, method="naive_bayes", trControl=ctrl)
m_lda <- train(Species~., data=iris, method="lda", trControl=ctrl)

resultados <- resamples(list(KNN=m_knn, NB=m_nb, LDA=m_lda))
summary(resultados)
bwplot(resultados, metric="Accuracy") # boxplot de accuracy por modelo
dotplot(resultados, metric="Accuracy") # dotplot alternativo

# Test estadístico de diferencia
diffs <- diff(resultados)
summary(diffs) # p-valores de comparación entre modelos

```

Python — sklearn evaluación completa

```

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.model_selection import cross_val_score, StratifiedKFold, cross_validate
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

X, y = load_iris(return_X_y=True)
names = load_iris().target_names

# LDA
lda = LinearDiscriminantAnalysis()
lda.fit(X, y) # en todos los datos para visualización
X_lda = lda.transform(X) # proyectar a espacio LDA

# Visualizar proyección
fig, ax = plt.subplots(figsize=(8,6))
for i, name in enumerate(names):
    mask = y == i
    ax.scatter(X_lda[mask,0], X_lda[mask,1], label=name, alpha=0.7, s=60)
ax.set_xlabel('LD1'); ax.set_ylabel('LD2')
ax.set_title('Proyección LDA - iris'); ax.legend()
plt.tight_layout()

# EVALUACIÓN RIGUROSA
# -----
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

modelos = {
    'LDA': LinearDiscriminantAnalysis(),
    'QDA': QuadraticDiscriminantAnalysis(),
    'KNN': Pipeline([('scaler', StandardScaler()),
                     ('knn', KNeighborsClassifier(n_neighbors=5))]),
    'NaiveBayes': GaussianNB()
}

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

resultados = {}
for nombre, modelo in modelos.items():

```

```
scores = cross_val_score(modelo, X, y, cv=cv, scoring='accuracy')
resultados[nombre] = scores
print(f"{nombre}: {scores.mean():.4f} ± {scores.std():.4f}")

# Visualización comparativa
res_df = pd.DataFrame(resultados)
res_df.plot(kind='box', figsize=(8,5))
plt.title('Comparación de modelos – Accuracy (10-fold CV)')
plt.ylabel('Accuracy'); plt.xlabel('Modelo')
plt.grid(True, alpha=0.3)
plt.tight_layout()
```